

DNH — Plan triển khai Go-Live: Incremental Alert Pipeline + Fix Gaps

Dùng file này làm prompt cho Claude Code trong repo [danglvmcna/DNHxLVD](#).
Thực hiện tuần tự theo từng Phase, mỗi Phase có Acceptance Criteria rõ ràng — không sang Phase tiếp theo nếu Phase hiện tại chưa pass criteria.

Bối cảnh (để Claude Code hiểu trước khi sửa code)

- Repo: [ai_agent/chatbot.py](#) (NL2SQL chatbot), [src/alerts.py](#) + [src/notifier.py](#) (alert engine), [main.py](#) (scheduler loop), [backend/main.py](#) (FastAPI middleware cho web UI), [frontend/](#) (vanilla JS chat UI).
 - Nguồn dữ liệu: Bravo (OTC) và DMS (ETC) là 2 hệ thống ERP/CRM nguồn, **chỉ được đọc, tuyệt đối không ghi/không tạo trigger trên DB của Bravo/DMS**.
 - Đích: SQL Server on-prem (3 layer raw/staging/mart). Không dùng Supabase/CLOUD_DB_URL nữa cho bản go-live (cần xác nhận với DNH nếu vẫn còn dùng song song thì giữ lại nhưng đánh dấu deprecated).
 - Quyết định đã chốt: **Chatbot chỉ chạy trên web UI, KHÔNG làm Teams Bot hỏi-đáp nữa**. Teams chỉ nhận alert real-time qua Incoming Webhook.
 - Quyết định độ trễ real-time: **polling incremental mỗi 5-10 phút**, không dùng CDC/trigger ở giai đoạn go-live này (lý do: tránh động vào DB nguồn Bravo/DMS, tránh phụ thuộc vendor).
-

Phase 0 — Khảo sát trước khi code (BẮT BUỘC làm trước, đừng code mù)

Việc cần làm:

1. Kiểm tra từng bảng nguồn liên quan (hóa đơn, công nợ, tồn kho, KPI) trong Bravo/DMS xem có cột nào theo dõi thay đổi không: [ModifiedDate](#), [UpdatedAt](#), [rowversion](#), [timestamp](#) (SQL Server built-in rowversion type), hoặc số tăng dần ([ID IDENTITY](#), [SaveDate](#)).
2. Với mỗi bảng, ghi lại: tên cột thay đổi (nếu có), kiểu dữ liệu, có index sẵn trên cột đó không (nếu chưa có index, incremental query sẽ chậm — cần xin phép DBA tạo index, đọc-only nên an toàn).

3. Output của Phase này: 1 bảng ánh xạ (mapping table) dạng:

Bảng nguồn	Cột tracking	Kiểu dữ liệu	Có index?	Ghi chú
brv_hoadonhdr	SaveDate	datetime	có/không	...
brv_hoadonct	...	...	...	...
dms_khachhang	...	...	...	...
receivable_detail (mart, tự build)	period	text	...	dùng logic riêng, xem Phase 2
inventory (mart, tự build)	...	...	...	...

4. Nếu bảng nào **không có cột tracking nào** → ghi rõ, xử lý riêng ở Phase 2 (full-scan có giới hạn phạm vi, ví dụ chỉ scan dữ liệu trong N ngày gần nhất thay vì toàn bộ lịch sử).

Acceptance Criteria: Có bảng mapping đầy đủ, review bởi Đăng, trước khi sang Phase 1.

Phase 1 — Fix bug chặn go-live trong code hiện tại (không phụ thuộc ai, làm ngay)

1.1. Sửa `main.py` — đang gọi sai hàm cảnh báo

File: `main.py`

Hiện tại vòng lặp `while True` trong `main()` chỉ gọi:

```
run_alert_checks(erp_engine, crm_engine) # <- đây là bản MOCK, chạy trên mock_source/erp.db, crm.db
```

Cần sửa thành: gọi đúng các hàm cảnh báo thật đọc từ dữ liệu DNH thật:

```
from src.alerts import run_smart_business_alerts, run_sales_kpi_insights_alert
```

Trong vòng lặp chính, thay thế / bổ sung:

`run_smart_business_alerts()`

`run_sales_kpi_insights_alert()`

- Xóa hẳn hoặc comment rõ ràng phần gọi `run_alert_checks(erp_engine, crm_engine)` (bản mock), hoặc giữ lại có flag `config['environment'] == 'local'` để dev vẫn test được bằng mock data, nhưng **production phải luôn chạy bản thật**.
- Đảm bảo `erp_engine, crm_engine` không còn được dùng nếu bỏ hẳn mock — dọn import thừa.

Acceptance Criteria: Chạy `python main.py --once` → log in ra rõ ràng đang chạy `run_smart_business_alerts()` + `run_sales_kpi_insights_alert()`, không còn nhắc tới mock ERP/CRM.

1.2. Bổ sung 2 trigger còn thiếu trong `src/alerts.py`

Hiện có 3 trigger thật (nợ quá hạn tuyệt đối >10tr, cháy kho <1 tháng, KPI <60%). **Thiếu 2 trigger đã chốt với DNH:**

(a) Doanh thu giảm >20% so với kỳ trước

- Viết hàm mới `check_revenue_drop_alert()` trong `src/alerts.py`.
- Logic: lấy tổng doanh thu (Amount9, loại trừ dòng CTKM) theo kỳ hiện tại (period mới nhất) và kỳ liền trước (period trước đó theo `_latest_period_key` logic đã có trong `chatbot.py` — tái sử dụng hàm đó, đừng viết lại logic parse period).
- So sánh: nếu $(\text{doanh_thu_ky_truoc} - \text{doanh_thu_ky_nay}) / \text{doanh_thu_ky_truoc} > 0.20$ → trigger alert CRITICAL, gửi Teams + xem xét gửi cả email tùy mức độ nghiêm trọng (thảo luận với DNH: gửi kênh nào).
- Cần nhắc: doanh thu nên tính theo **cùng kỳ** (so với kỳ trước liền kề, ví dụ tháng N vs tháng N-1) hay so với **cùng kỳ năm trước** (tháng N năm nay vs tháng N năm ngoái, tránh nhiễu do mùa vụ)? → Cần hỏi DNH cách tính chính xác đã thống nhất trong `MCNA_DNH_ProjectPlan_v3.docx`, đừng tự suy diễn nếu tài liệu ghi rõ.

(b) Nợ vượt hạn mức tín dụng (credit limit)

- Viết hàm mới `check_credit_limit_exceeded_alert()`.
- Cần xác nhận: bảng `receivable_detail` hoặc bảng khách hàng (`dms_khachhang`) có cột hạn mức tín dụng (credit limit) theo từng khách hàng không? Nếu **chưa có cột này trong schema hiện tại**, đây là gap dữ liệu, không phải gap code — cần hỏi DNH

xem hạn mức tín dụng lưu ở đâu (có thể ở Bravo/DMS chưa được đưa vào staging/mart) trước khi viết được logic này.

- Logic dự kiến (khi đã có cột): so sánh `balance_end` (dư nợ hiện tại) với `credit_limit` của khách hàng đó, nếu vượt → trigger CRITICAL, liệt kê danh sách khách hàng vượt hạn mức + số tiền vượt.

Acceptance Criteria: Cả 4 trigger (nợ quá hạn, cháy kho, KPI thấp, doanh thu giảm >20%, nợ vượt hạn mức — tổng 5 vì có thêm), mỗi trigger:

- Có cooldown hợp lý (dùng lại `should_send_alert/record_alert_sent` đã có).
- Có unit test đơn giản (dùng dữ liệu mẫu trong `dnh_intermediate.db` hoặc mock) xác nhận trigger bắn đúng khi vượt ngưỡng, không bắn khi dưới ngưỡng.

Phase 2 — Xây incremental sync layer (đáp ứng yêu cầu polling 5-10 phút)

2.1. Bảng state để track lần sync gần nhất

Tạo bảng mới trong SQL Server staging (hoặc SQLite intermediate nếu vẫn giữ giai đoạn transition):

```
CREATE TABLE sync_state (  
  
    source_table VARCHAR(100) PRIMARY KEY,  
  
    last_synced_at DATETIME NOT NULL,  
  
    last_synced_id BIGINT NULL -- dùng nếu bảng dùng ID tăng dần thay vì timestamp  
  
);
```

2.2. Viết hàm incremental fetch cho từng bảng nguồn

Dựa vào bảng mapping từ Phase 0:

- Nếu bảng có cột timestamp (`ModifiedDate/SaveDate`):

```
SELECT * FROM <bảng_nguồn>  
  
WHERE <cột_timestamp> > @last_synced_at
```

ORDER BY <cột_timestamp> ASC

- Nếu bảng có ID tăng dần nhưng không có timestamp:

```
SELECT * FROM <bảng_nguồn> WHERE <ID> > @last_synced_id ORDER BY <ID>
ASC
```

- Nếu bảng **không có cột tracking nào** (theo Phase 0 note): giới hạn scan trong cửa sổ thời gian gần nhất (ví dụ N ngày gần đây dựa vào cột ngày chứng từ **DocDate** nếu có), KHÔNG full scan toàn bộ lịch sử mỗi 5-10 phút.
- Sau mỗi lần fetch thành công: update **sync_state.last_synced_at** (hoặc **last_synced_id**) = giá trị lớn nhất vừa lấy được (không dùng **NOW()** của hệ thống mình, tránh lệch giờ giữa 2 server).

2.3. Idempotent upsert vào staging/mart

- Dùng **MERGE** (T-SQL) hoặc **INSERT ... ON CONFLICT** tùy DB đích, dựa trên khóa nghiệp vụ (invoice number, customer code + period, v.v.) để tránh trùng lặp khi retry.
- Áp dụng lại đúng các quy tắc join/dedup đã có trong **ai_agent/chatbot.py** system prompt (join theo channel riêng OTC/ETC rồi UNION, loại trừ dòng CTKM khi tính doanh số...) — đừng viết lại logic nghiệp vụ khác với những gì đã thống nhất và encode trong chatbot prompt.

2.4. Scheduler cho sync job

- Tần suất: 5-10 phút (đề xuất bắt đầu 10 phút cho go-live, giảm xuống 5 phút sau nếu ổn định và tải nguồn cho phép — đo thời gian chạy 1 lần sync mất bao lâu trước khi rút ngắn chu kỳ).
- Chạy sync trước, xong mới chạy **run_smart_business_alerts()** / **run_sales_kpi_insights_alert()** (đảm bảo alert luôn đọc dữ liệu vừa mới sync, không đọc dữ liệu cũ hơn 1 chu kỳ).
- Log rõ: thời gian bắt đầu/kết thúc sync, số dòng mới lấy được mỗi bảng, lỗi nếu có (không được để lỗi 1 bảng làm crash toàn bộ job — try/except riêng từng bảng, log lỗi, tiếp tục bảng khác).

Acceptance Criteria:

- Chạy sync job 2 lần liên tiếp cách nhau vài phút với dữ liệu test có insert thêm ở giữa → lần 2 chỉ lấy đúng phần mới, không lấy lại dữ liệu cũ.
- Tắt job giữa chừng (kill process) rồi chạy lại → không bị mất dữ liệu, không bị trùng lặp (idempotent) — test bằng cách đếm số dòng trước/sau.
- Đo thời gian chạy 1 chu kỳ sync đầy đủ, phải nhỏ hơn nhiều so với chu kỳ 5-10 phút (nếu gần bằng hoặc vượt, phải tối ưu query hoặc giãn chu kỳ ra thêm).

Phase 3 — Chạy nền ổn định (production-grade scheduler)

- Đóng gói `main.py` (sync job + alert job) thành Windows Service dùng NSSM hoặc Task Scheduler (chạy lúc boot, tự restart khi crash) — tham khảo cách đã làm với `sync_daemon.py` trong `scripts/register_sync_service.bat / start_sync_service.bat`, viết bản tương tự cho `main.py`.
- Thêm health-check đơn giản: ghi timestamp lần chạy thành công cuối cùng vào 1 file/bảng, có thể dùng để cảnh báo riêng nếu job bị đứng quá lâu (ví dụ không chạy được >30 phút).

Acceptance Criteria: Restart máy chủ → service tự chạy lại không cần thao tác thủ công. Kill process → service tự khởi động lại (nếu dùng NSSM với auto-restart).

Phase 4 — Notification channels (Teams + Outlook)

- Xác nhận `TEAMS_WEBHOOK_URL` thật đã cấu hình trong `.env` (không phải placeholder).
- Với Outlook: thử gửi thật qua SMTP hiện tại trước — nếu fail do basic auth bị chặn (lỗi auth từ `smtpplib`), chuyển sang implement `send_email_via_graph_api()` dùng Microsoft Graph `POST /users/{id}/sendMail` với OAuth2 client credentials flow — cần `MS_GRAPH_CLIENT_ID`, `MS_GRAPH_CLIENT_SECRET`, `MS_GRAPH_TENANT_ID` (xin từ IT DNH, cần App Registration có quyền `Mail.Send` application permission, admin consent).
- Viết cả 2 đường (`smtp` và `graph`), chọn qua config flag `email.provider: smtp | graph`, để dễ chuyển đổi khi biết chính xác tenant DNH hỗ trợ cái nào.

Acceptance Criteria: Gửi thử 1 email report thật đến hộp thư test của Đăng, xác nhận nhận được, format hiển thị đúng trên Outlook desktop và Outlook web.

Phase 5 — Web UI hardening (vì không còn Teams Bot, web là kênh chatbot duy nhất)

5.1. Dọn code Teams Bot không cần nữa

- Xóa hoặc di chuyển vào thư mục `_deprecated/`: `src/teams_bot.py`, `manifest.json`.

- Trong `backend/main.py`: xóa `from botbuilder.schema import Activity` và `from src.teams_bot import ADAPTER, dnh_bot`, xóa route `/api/messages` nếu có.
- Xóa `botbuilder-core`, `botbuilder-schema` khỏi `backend/requirements.txt`.

5.2. Thay auth demo bằng auth thật

- Bỏ `USERS = {...}` hardcode plaintext trong `backend/main.py`.
- Dùng bcrypt hash mật khẩu (lưu trong bảng DB, không hardcode trong source).
- Token: dùng JWT ký bằng secret key (lưu trong `.env`, không commit), có `exp` claim (hết hạn sau N giờ), verify chữ ký thật trong `verify_token()` thay vì chỉ check string nằm trong list cứng.
- Khuyến nghị dài hạn (không bắt buộc go-live): tích hợp SSO Azure AD vì DNH đã dùng M365 sẵn — an toàn hơn tự quản mật khẩu cho C-level, nhưng có thể để Phase sau nếu timeline gấp.

5.3. Siết CORS

- Đổi `allow_origins=["*"]` thành domain cụ thể sẽ host frontend (ví dụ nội bộ DNH).

5.4. Chốt 1 frontend

- Quyết định dùng `frontend/` (vanilla, đã chạy được) làm bản go-live chính thức.
- Archive hoặc xóa `frontend/nextjs_chat_skeleton/` nếu không tiếp tục phát triển, tránh gây nhầm lẫn có 2 bản song song.

Acceptance Criteria:

- Login bằng tài khoản thật (không phải hardcode) → nhận JWT hợp lệ, hết hạn đúng thời gian cấu hình.
- Gọi API dashboard/chatbot bằng token hết hạn → trả về 401.
- Gọi từ domain lạ (không phải domain đã whitelist CORS) → bị chặn bởi trình duyệt.
- Không còn file/route nào liên quan Teams Bot trong repo.

Checklist tổng hợp trước khi bấm go-live

- ☐ Phase 0: có bảng mapping cột tracking cho từng bảng nguồn, đã review.
- ☐ Phase 1: `main.py` gọi đúng hàm alert thật; đủ 5 trigger (nợ quá hạn, cháy kho, KPI thấp, doanh thu giảm >20%, nợ vượt hạn mức) — trigger (b) cần xác nhận có cột credit limit trước.
- ☐ Phase 2: incremental sync chạy đúng, idempotent, đo thời gian chạy < chu kỳ đặt ra.

- ☐ Phase 3: chạy như Windows Service, tự phục hồi khi crash/reboot.
- ☐ Phase 4: Teams webhook thật hoạt động; Outlook gửi được qua SMTP hoặc Graph API (đã xác nhận với IT DNH phương án nào dùng được).
- ☐ Phase 5: web UI có auth thật, CORS siết chặt, không còn code Teams Bot thừa, chỉ 1 frontend.
- ☐ Đã chốt với DNH: giờ chính xác daily digest (7h sáng / sau giờ hành chính / giờ khác).
- ☐ Đã chốt với DNH: nguồn dữ liệu cuối cùng là SQL Server on-prem, có connection string thật.